LABORATORIO 1: INTRODUCCIÓN A HASKELL Y PROGRAMACIÓN FUNCIONAL

Gabriel Andrés Camargo Barón

---

1. EJERCICIO 1 - FUNDAMENTOS BÁSICOS Y TIPOS

a) Consultas de tipo con :t

- `:t True` → `Bool`
- `:t "Hola mundo"` → `[Char]`
- `:t [1,2,3]` → `Num a => [a]`

b) Pregunta teórica: Fuertemente tipado y evaluación perezosa

- **Fuertemente tipado:** Haskell verifica que los tipos de datos sean compatibles antes de ejecutar el programa. No permite operar un entero con un texto ni hace conversiones automáticas.
- **Evaluación perezosa:** Haskell no calcula ninguna expresión hasta que el resultado sea estrictamente necesario. Esto evita procesar datos innecesarios y optimiza la memoria.

c) Diferencia entre Int y Float

`Int` se utiliza para representar números enteros exactos (sin decimales).
`Float` representa números reales con parte decimal (coma flotante).

d) Código Haskell (Ejercicio1.hs)

```
module Ejercicio1 where

listaConcatenada :: [Int]

listaConcatenada = [1,2,3] ++ [4,5]

doble :: Int → Int

doble x = x * 2
```

esMayorEdad :: Int → Bool

esMayorEdad edad = edad >= 18

areaCirculo :: Float → Float

areaCirculo radio = pi * (radio ^ 2)

Pruebas ejecutadas en consola para capturas de pantalla:

- `doble 5` → `10`
- `esMayorEdad 20` → `True`
- `areaCirculo 3` → `28.274334`

---

2. EJERCICIO 2 - LISTAS Y OPERACIONES BÁSICAS

a) Análisis de [1..10] y evaluación perezosa

La expresión `[1..10]` genera una secuencia del 1 al 10. Se relaciona con la evaluación perezosa porque Haskell crea los elementos únicamente cuando una función se los pide. Si se usa `take 3 [1..10]`, solo calcula los primeros 3 números e ignora el resto.

b) Código Haskell (Ejercicio2.hs)

```
module Ejercicio2 where

primerElemento :: [a] → a
primerElemento (x:_) = x
primerElemento []    = error "Lista vacía"

productoLista :: [Int] → Int
productoLista []     = 1
productoLista (x:xs) = x * productoLista xs
```

eliminarNegativos :: [Int] → [Int]
eliminarNegativos [] = []
eliminarNegativos (x:xs)
| x >= 0   = x : eliminarNegativos xs
| otherwise = eliminarNegativos xs

invertir :: [a] → [a]
invertir []   = []
invertir (x:xs) = invertir xs ++ [x]

Pruebas ejecutadas en consola para capturas de pantalla:

* `primerElemento [10, 20, 30]` → `10`

* `productoLista [1, 2, 3, 4]` → `24`

* `eliminarNegativos [-3, 5, -1, 8]` → `[5, 8]`

* `invertir [1, 2, 3]` → `[3, 2, 1]`

---

3. EJERCICIO 3 - DEFINICIÓN DE FUNCIONES: COMPOSICIÓN Y CASOS

Código Haskell (Ejercicio3.hs)

```haskell

module Ejercicio3 where

cuadrado :: Int → Int

cuadrado x = x * x

suma :: Int → Int → Int
```

```
suma x y = x + y

sumaCuadrados :: Int → Int → Int

sumaCuadrados x y = suma (cuadrado x) (cuadrado y)

duplicar :: Int → Int

duplicar x = x * 2

dobleLista :: [Int] → [Int]

dobleLista xs = map duplicar xs

clasificarNota :: Int → String

clasificarNota nota

    | nota < 3              = "Reprobado"

    | nota >= 3 && nota < 4 = "En recuperación"

    | otherwise             = "Aprobado"

fibonacci :: Int → Int

fibonacci 0 = 0

fibonacci 1 = 1

fibonacci n

    | n > 1     = fibonacci (n - 1) + fibonacci (n - 2)

    | otherwise = error "Número inválido"
```

Pruebas ejecutadas en consola para capturas de pantalla:

- `sumaCuadrados 3 4` → `25`
- `dobleLista [1, 2, 3, 4]` → `[2, 4, 6, 8]`
- `clasificarNota 4` → `"Aprobado"`
- `fibonacci 6` → `8`

---

4. EJERCICIO 4 - EVALUACIÓN FUNCIONAL

a) Expresión: `take 5 (map (*2) [1..])`

- **Predicción:** Multiplica la lista infinita por 2 y toma los primeros 5. Resultado: `[2,4,6,8,10]`.
- **Resultado en consola:** `[2,4,6,8,10]`.
- **Análisis:** No genera un ciclo infinito porque `take 5` detiene la solicitud de datos una vez obtiene los 5 valores que necesita.

b) Evaluación paso a paso de `misterio (5+7) (4*3)`

Dada la función:misterio x y = if x > 10 then y * 2 else x + y

1. **Paso 1:** Evaluar x → `5 + 7 = 12`
2. **Paso 2:** Evaluar condición → `12 > 10` es `True`
3. **Paso 3:** Tomar rama `then` → `y * 2`
4. **Paso 4:** Evaluar y y resolver → `(4 * 3) * 2 = 12 * 2 = 24`

---

5. EJERCICIO 5 - EXPRESIONES LAMBDA

Código Haskell (Ejercicio5.hs)

```
module Ejercicio5 where

dobleLambda = \x → x * 2

cuboLambda = \x → x ^ 3
```

mitadLambda = \x → x / 2

esPositivoLambda = \x → x > 0

enIntervaloLambda = \x → x >= 10 && x <= 20

sumaLambda = \x y → x + y

multiplicacionLambda = \x y → x * y

potenciaLambda = \x y → x ^ y

maximoLambda = \x y → if x >= y then x else y

sonIgualesLambda = \x y → x == y

Pruebas ejecutadas en consola para capturas de pantalla:

- `dobleLambda 5` → `10`
- `cuboLambda 3` → `27`
- `sumaLambda 4 6` → `10`

---

6. RETROSPECTIVA

1. **Tiempo total invertido:** 4 horas hombre.
2. **Estado actual:** Completado al 100%. Todos los ejercicios probados.
3. **Mayor logro:** Entender el funcionamiento de la evaluación perezosa con listas infinitas.
4. **Mayor problema técnico:** Entender la carga de módulos en el compilador. Se resolvió usando las funciones adecuadamente en la consola interactiva.
5. **Trabajo en equipo y mejora:** Se organizó bien el desarrollo de cada punto. Como mejora se profundizará más en el uso de la terminal de comandos.

6. **Referencias:**
    - Lipovača, M. (2011). *Learn You a Haskell for Great Good!*. No Starch Press.
    - Haskell Documentation. https://www.haskell.org/documentation/



